

UOS ASSISTANT - Intelligent University Voice Assistant

Project Advisor

- Dr. Muhammad Saad Razzaq (Internal)

Project Manager

- Dr. Muhammad Ilyas

Project Team

<i>Muhammad Asmaad Saeed</i>	<i>BSCS51F22S037</i>	<i>Team Lead</i>
<i>Laiba Siraj</i>	BSCS51F22S046	Team Member
<i>Fatima Zahid</i>	BSCS51F22S010	Team Member
<i>Areeba Rehman</i>	BSCS51F22S039	Team Member

Submission Date

- 8th October 2025

Table of Contents

<i>1. Abstract.....</i>	<i>3</i>
<i>2. Background and Justification</i>	<i>3</i>
<i>3. Project Methodology</i>	<i>3</i>
<i>3.2 System Architecture and Workflow.....</i>	<i>4</i>
<i>3.3 Server Infrastructure and DevOps Tools.....</i>	<i>6</i>
<i>4. Project Scope.....</i>	<i>9</i>
<i>Included Functionalities:</i>	<i>9</i>
<i>Excluded Functionalities:.....</i>	<i>9</i>
<i>5. High level Project Plan</i>	<i>10</i>
<i>6. References</i>	<i>6</i>

1. Abstract

The **UOS ASSISTANT** is an Urdu-speaking virtual assistant developed for the University of Sargodha (UOS). It aims to modernize and automate the university’s telephone-based support system. Unlike existing English-based solutions, it enables natural Urdu communication to remove language barriers and improve accessibility for students, especially those calling from remote areas of Pakistan.

Human-operated call centers face high call volumes, long waiting times, and inconsistent service quality. UOS Assistant addresses these challenges through an **AI-powered voice pipeline**: Automatic Speech Recognition (ASR) using Google Speech-to-Text with Urdu dialect support, an Agent Controller that orchestrates query processing, OpenAI’s text-embedding-3-small model for semantic embedding, a PostgreSQL-backed vector store for context retrieval, OpenAI GPT-4o Mini for structured response generation, and Google Text-to-Speech (TTS) for natural voice delivery.

The system processes user voice queries end-to-end from microphone input, through the Google STT engine, to the Agent Controller which embeds the query, performs vector search on PostgreSQL, combines the retrieved context with the user question and a structured prompt, sends it to GPT-4o Mini for response generation, and finally delivers the answer via Google TTS to the caller. The backend infrastructure runs on a Linux server accessed via SSH (PuTTY), with PostgreSQL managed through DBeaver and files transferred via FileZilla FTP. The agent handles routine queries automatically and escalates complex cases to human operators.

2. Background and Justification

In today’s digital landscape, customer support plays a central role in shaping user experience and organizational reputation. However, traditional inbound call centers are burdened with long wait times and inconsistent service quality. These challenges are more pronounced in Pakistan, where many users communicate in Urdu or a mix of Urdu-English (code-switching), yet most automated systems remain English-centric, creating a significant language and accessibility gap. [1]

With users expecting real-time, 24/7, and personalized support, organizations are under increasing pressure to optimize resources while maintaining service quality. Existing solutions such as AutoCalls.ai offer generic call automation tools but are not sufficiently adaptable to specialized contexts such as Urdu-first interaction, code-switch handling, or localized Pakistani service delivery. [2–4]

The *University of Sargodha receives hundreds of repetitive inbound calls* daily regarding admissions, fee structures, program eligibility, and course information. Staff members are burdened with answering identical queries, while callers face long waiting times especially outside office hours. There is a clear need for an intelligent, always-available system that can handle these queries in the caller’s preferred language.

This project proposes UOS Assistant: a native Urdu-first AI calling agent built on a Retrieval-Augmented Generation (RAG) architecture. The system integrates Google ASR, a PostgreSQL

vector store populated with university documents, OpenAI embeddings and GPT-4o Mini for response generation, and Google TTS for voice output all orchestrated through a central Agent Controller deployed on a Linux server infrastructure.

3. Project Methodology

The development of UOS Assistant follows a structured and iterative Agile Software Development Life Cycle (SDLC). The project is organized into short sprints, each producing a working, testable increment of the system.

Requirement Analysis

- Collecting functional and non-functional requirements for university call handling.
- Identifying use cases: fee queries, admission inquiries, program eligibility, course information, and academic calendar updates.
- Defining data sources: UOS Prospectus PDFs, fee structure documents, and CSV data files containing admission information.

System and Architectural Design

- Designing the end-to-end voice pipeline: UI → Google STT → Agent Controller → Embedding → Vector Search → GPT-4o Mini → Google TTS → UI.
- Defining the RAG ingestion pipeline: data files (CSV/PDF) → chunking → OpenAI text-embedding-3-small → PostgreSQL vector store.
- Designing the admission_document database schema: id | content | embedding | category | source_file.
- Designing the call assistant backend (CABE) using Java Spring Boot with Maven for telephony integration.
- Creating UML diagrams: use case, sequence, and component diagrams.

Implementation and Integration

- Python and FastAPI for the main AI backend and Agent Controller.
- Java Spring Boot (Maven) for the telephony Call Assistant Backend (CABE).
- PostgreSQL with pgvector extension as the vector database, containerized via Docker.
- OpenAI API for embeddings (text-embedding-3-small) and response generation (GPT-4o Mini).
- Google Cloud APIs for Speech-to-Text (STT) and Text-to-Speech (TTS).
- Linux server (Ubuntu) as the deployment environment, accessed via SSH using PuTTY.
- FileZilla FTP client for uploading/downloading project files to and from the server.
- DBeaver as the database client for accessing and managing PostgreSQL on the server.
- Microsoft Azure for cloud infrastructure and deployment.

Testing and Validation

- Unit testing, integration testing, and end-to-end testing for all modules.
- Evaluating speech recognition accuracy, embedding quality, retrieval relevance (top-K), and response latency.

- Testing under real call conditions using telephony APIs (Twilio/SignalWire).

Deployment and Continuous Improvement

- Deploying on Microsoft Azure using Docker containers for the PostgreSQL vector database and microservices.
- Monitoring system performance, tracking call analytics, and retraining models based on feedback.
- Iterating prompt engineering, embedding quality, and retrieval accuracy based on real usage data.

3.2. System Architecture and Workflow

UOS Assistant is built on a clean two-layer architecture: *a Frontend UI layer and a Backend processing pipeline*, orchestrated by a central Agent Controller. The complete request-response workflow is described below.

3.2.1 End-to-End Request-Response Flow

Step 1 - Frontend: Voice Input via UI

The user interacts through the UOS Assistant application (web and Android app). The workflow begins at the UI:

1. The user speaks a query in Urdu or English into the device microphone.
2. The audio stream is sent to the Google Speech-to-Text (STT) engine, configured with Urdu dialect support. The STT engine transcribes the spoken audio to text, handling code-switching between Urdu and English.
3. The transcribed text query is forwarded from the UI to the backend Agent Controller.

Step 2 - Backend: Agent Controller and Processing Pipeline

The Agent Controller is the central orchestration layer of the backend. It manages all message routing, state, and context:

4. The Agent Controller receives the transcribed message and initiates the processing pipeline.
5. Embedding Module: The query text is passed to the OpenAI text-embedding-3-small model, which converts it into a 1536-dimensional vector representation.
6. Vector Search (PostgreSQL + pgvector): The query embedding is matched against pre-stored document embeddings in the PostgreSQL vector database using cosine similarity. The system retrieves the top-K most relevant document chunks from the `admission_document` table.
7. Prompt Construction: The Agent Controller combines three components into a single LLM input: the structured system prompt, the user's original question, and the retrieved context chunks.
8. Response Generation (GPT-4o Mini): The combined prompt+question+context is sent to OpenAI GPT-4o Mini, which generates a structured, context-aware response message.

9. The structured response (JSON containing response text) is returned to the UI layer.

Step 3 - Frontend: Voice Response Delivery

10. The response text is extracted from the JSON response and passed to the Google Text-to-Speech (TTS) engine.
11. Google TTS synthesizes the text into natural speech using WaveNet voices, supporting Urdu and English output.
12. The spoken audio response is played to the user through the device speaker, and the response text is simultaneously displayed on screen.

3.2.2 Call Assistant Backend (CABE)

A dedicated Call Assistant Backend (CABE) is built using Java Spring Boot with Maven (MVM). CABE handles the telephony integration layer managing inbound call routing, session lifecycle, and escalation to human operators when the AI cannot resolve a query. CABE communicates with the main backend via REST APIs.

➤ Architecture Summary:

Layer	Dir.	Component / Action
UI (Frontend)	→	User speaks → Google STT (Urdu) → Transcribed text
UI (Frontend)	→	Text query forwarded to Agent Controller (Backend)
Backend	→	Agent Controller → OpenAI text-embedding-3-small → 1536-dim vector
Backend	→	Vector Search on PostgreSQL + pgvector → Top-K matching chunks returned
Backend	→	Prompt + Question + Context combined → sent to GPT-4o Mini
Backend	→	GPT-4o Mini generates structured response message (JSON)
UI (Frontend)	←	response_text → Google TTS → Voice played + Text displayed on screen

➤ Technology Stack and Resources:

Category	Technology / Tool	Purpose
Speech Recognition	Google STT API	Urdu/English speech-to-text with dialect support

Text-to-Speech	Google TTS (WaveNet)	Natural voice response synthesis in Urdu/English
LLM	OpenAI GPT-4o Mini	Context-aware structured response generation
Embeddings	OpenAI text-embedding-3-small	Query and document vectorization (1536 dimensions)
Vector Store	PostgreSQL + pgvector	Vector similarity search for RAG retrieval
Main Backend	Python + FastAPI	Agent Controller, embedding, and RAG pipeline
Call Backend (CABE)	Java Spring Boot (Maven)	Telephony integration, call routing, escalation
Containerization	Docker	PostgreSQL vector DB container and microservices
Cloud Platform	Microsoft Azure	Production server deployment and scaling
Server OS	Linux (Ubuntu)	Server-side operating environment
SSH Client	PuTTY	SSH access to the Linux server for remote management
FTP Client	FileZilla	Uploading/downloading files to and from the server
DB Client	DBeaver	Accessing and managing PostgreSQL database on server
Frontend / App	Android App	UOS Assistant user-facing web and mobile interface
Data Sources	UOS Prospectus, Fee Structure PDFs, CSV files	University knowledge base for RAG ingestion

➤ Admission Document Schema:

The PostgreSQL vector store uses a single admission_document table with the following structure:

id	content	embedding	category	source_file
UUID / Serial	Original text chunk	1536-dim vector (pgvector)	e.g. Fees, Admissions	Source document name

3.3. Server Infrastructure and DevOps Tools

UOS Assistant is deployed on a Linux (Ubuntu) server hosted on Microsoft Azure. The following tools are used to manage, access, and maintain the server infrastructure:

1. PuTTY - SSH Client

PuTTY is used as the primary SSH client for establishing secure remote connections to the Linux server. It is used by the development team to:

- Access the server command line for deploying and restarting backend services.
- Monitor system logs and process status.
- Trigger the RAG ingestion pipeline when new data is uploaded.
- Manage Docker containers running the PostgreSQL vector database.

2. FileZilla - FTP Client

FileZilla is used as the FTP/SFTP client for file transfer between local development machines and the remote Linux server. It is used to:

- Upload new university data files (CSV, PDF) to the server for RAG ingestion.
- Download log files, output data, and processed results from the server.
- Transfer updated backend code and configuration files during development iterations.

3. DBeaver - Database Client

DBeaver is used as the database management client for accessing the PostgreSQL vector database running on the server. It is used to:

- Inspect and verify the admission_document table after RAG ingestion runs.
- Run SQL queries to check embedding counts, data quality, and retrieval test queries.
- Monitor database size and performance metrics.
- Manage database schema changes and backups.

4. Docker - Containerization

The PostgreSQL + pgvector database is deployed as a Docker container, ensuring a consistent and isolated runtime environment. Docker allows the team to spin up, restart, or migrate the vector database without affecting other server components. All microservices (Python FastAPI backend, Java CABB) are also containerized for consistent deployment.

4. Project Scope

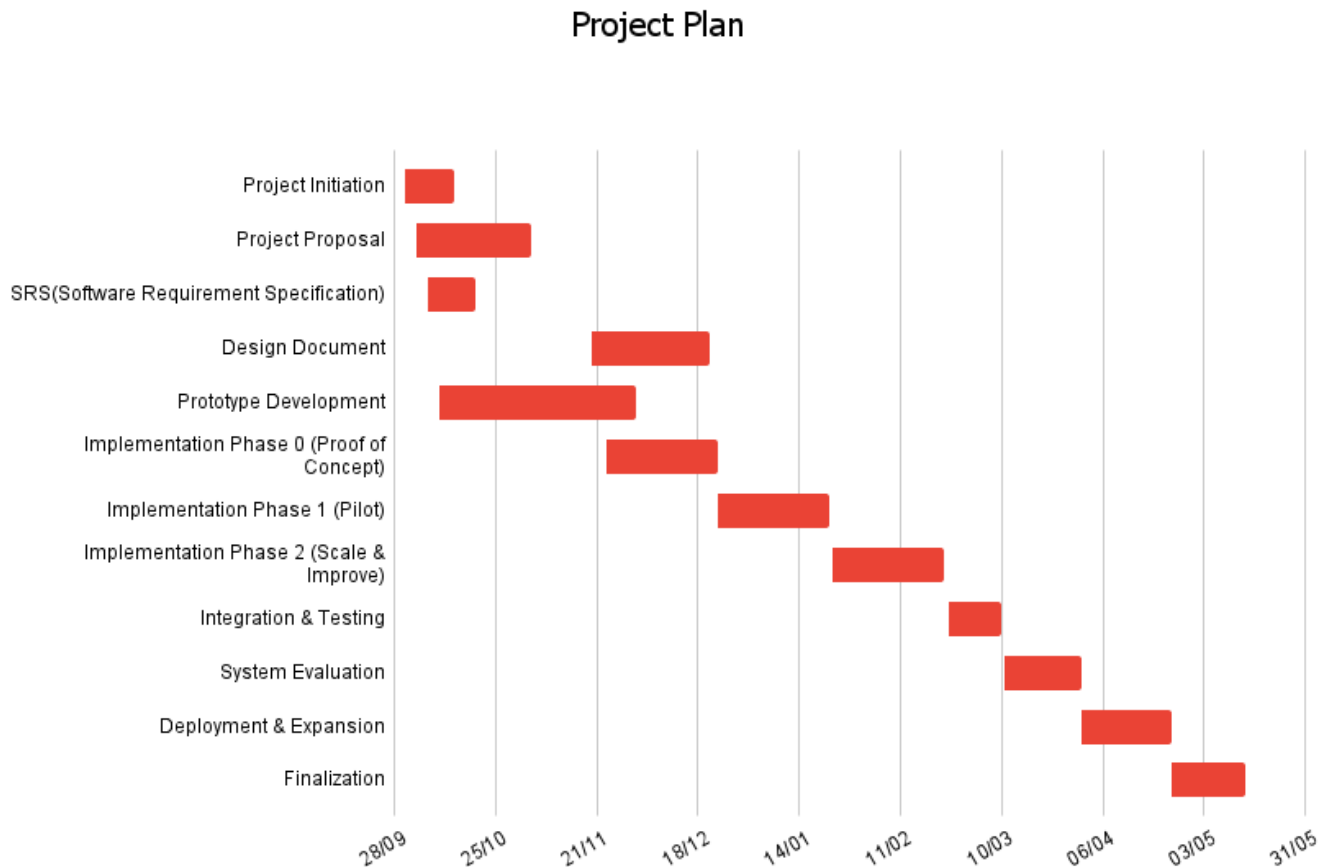
Included Functionalities:

- Google STT with Urdu dialect support for accurate speech transcription including Urdu-English code-switching.
- Agent Controller orchestrating the full query-to-response pipeline.
- Semantic retrieval using OpenAI text-embedding-3-small and PostgreSQL pgvector similarity search.
- Context-aware response generation using OpenAI GPT-4o Mini with structured prompt construction (prompt + question + context).
- Supports bilingual communication: Urdu, English, and mixed Urdu-English.
- Retrieves university information: admissions, fee structures, program eligibility, courses, and departmental contacts.
- UOS Assistant web and Android application as the user-facing interface.
- Linux server deployment on Azure managed via PuTTY (SSH), FileZilla (FTP), and DBeaver (DB).

Excluded Functionalities:

- Outbound or promotional calling.
- Video or chat-based communication channels.
- Integration with social media or non-telephony platforms.
- External support for non-verbal communication modes (email, text chat).

5.High level Project Plan



References

- [1] W. Bank, "Pakistan Development Update-Reimagining A Digital Pakistan," World Bank, April 2025. [Online]. Available: <https://thedocs.worldbank.org/en/doc/e414b36ae736660edf8f0f3cb597b1e9-0310012025/pakistan-development-update-reimagining-a-digital-pakistan-april-2025> .
- [2] H. B. Review, "Customer experience in the age of AI," Harvard Business Review, March 2022. [Online]. Available: <https://hbr.org/2022/03/customer-experience-in-the-age-of-ai>.
- [3] P. & Parloa, "Transforming customer service for a smarter, more human future.," Parloa, 2021. [Online]. Available: <https://www.parloa.com/customers/pwc-and-parloa-transforming-customer-service-for-a-smarter-more-human-future/>.
- [4] AutoCalls, "AutoCalls AI: Call flow builder.," AutoCalls, 2025. [Online]. Available: <https://autocalls.ai/>.

- [5] Gartner, "Market guide for conversational AI platforms.," Gartner Research, 2022. [Online]. Available: <https://www.gartner.com/en/documents/5332563>.
- [6] G. Cloud, ""Speech-to-Text documentation,"", Google, 2025. [Online]. Available: <https://cloud.google.com/speech-to-text/docs>.
- [7] Wikipedia, ""Natural language understanding,"", WIKIPEDIA, 2025. [Online]. Available: https://en.wikipedia.org/wiki/Natural_language_understanding.
- [8] LangChain, ""LangChain Docs,"", LangChain, October 2025. [Online]. Available: https://python.langchain.com/docs/integrations/retrievers/graph_rag/.
- [9] "Knowledge graph," WIKIPEDIA, 2025. [Online]. Available: https://en.wikipedia.org/wiki/Knowledge_graph.
- [10] 11ElevenLabs, ""ElevenLabs,"", 11ElevenLabs, 2025. [Online]. Available: <https://elevenlabs.io/>.
- [11] M. O. I. T. & TELECOMMUNICATION, "Pakistan Personal Data Protection Bill," MINISTRY OF INFORMATION TECHNOLOGY & TELECOMMUNICATION, 2023. [Online]. Available: <https://share.google/oUXqmt3btzePKP7hK>.
- [12] neo4j, "Neo4j documentation," neo4j, 2025. [Online]. Available: <https://neo4j.com/docs/>.
- [13] M. & K. A. & K. K. & T. Z. & A. H. Agro, "Code-Switching in End-to-End Automatic Speech Recognition: A Systematic Literature," 2025.
- [14] M. & O. J. & C. K. Kaleem, "Development of UMAIR the Urdu Conversational Agent for Customer Service.," 2014.
- [15] OpenAI, "GPT-5 Technical Report," 2025. [Online]. Available: <https://openai.com/>.